❸ The rest of the method remains the same.

❹ You must implement the Executed action to satisfy the interface.

❺ Derives from TypeFilter, which is used to fill dependencies using the DI container

❻ Passes the type EnsureRecipeExistsFilter as an argument to the base TypeFilter constructor

`EnsureRecipeExistsFilter` is a valid filter; you could use it on its own by adding it as a global filter (as global filters don't need to be attributes). But you can't use it directly by decorating controller classes and action methods, as it's not an attribute. That's where `EnsureRecipeExistsAttribute` comes in.

You can decorate your methods with `EnsureRecipeExistsAttribute` instead. This attribute inherits from `TypeFilterAttribute` and passes the `Type` of filter to create as an argument to the base constructor. This attribute acts as a *factory* for `EnsureRecipeExistsFilter` by implementing `IFilterFactory`.

When ASP.NET Core initially loads your app, it scans your actions and controllers, looking for filters and filter factories. It uses these to form a filter pipeline for every action in your app, as shown in figure 22.3.